

Informe Tarea 1

Felipe Valencia Godoy fvalenciag@usm.cl

Fernanda Huerta Sepulveda, fhuerta@usm.cl

Alonso Collao Orellana, acollao@usm.cl

Renato Antonio Zurich Vegas, rzurich@usm.cl

1. Parte 1

1.1. Diseño de la función lecturaSensor()

La función lecturaSensor() fue diseñada para simular la lectura de un sensor dentro del sistema e implementar un código modular que sea más amigable para el resto de programadores. Su funcionamiento consiste en generar un valor aleatorio dado por la función rand() dentro del rango [0,255], simulando una medición simulada.

Esta operación permite obtener valores enteros dentro del intervalo requerido.

El valor generado es almacenado en una variable temporal y posteriormente retornado para ser utilizado durante el proceso de lectura.

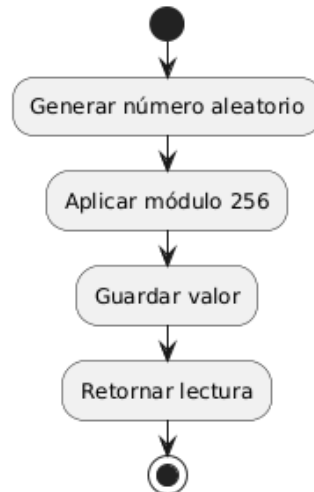
Además, se implementó el tipo enumerado "Accion", en el cual cada uno de sus valores representa una operación específica a ejecutar:

```
enum Accion {  
    INIT, READ, SHOW  
};
```

- INIT: inicialización de sensores
- READ: lectura de sensores
- SHOW: visualización de resultados

Utilizar un enumerado resulta útil pues permite utilizar la función usarSensores() dependiendo de las necesidades del usuario.

Diagrama de flujo (usando PlantUML)



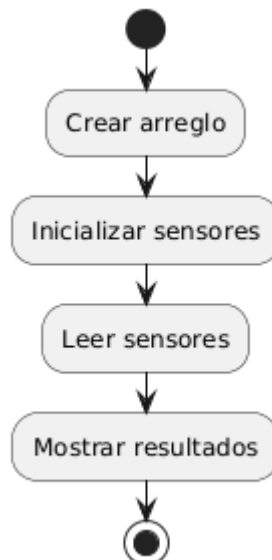
El diagrama representa el flujo interno de la función, desde la generación del valor aleatorio hasta su retorno.

1.2. Diseño completo de la nueva función main()

La nueva versión de la función main() fue diseñada para simplificar la estructura general del programa.

En la implementación original, todas las operaciones eran ejecutadas directamente mediante ciclos independientes.

Implementación original

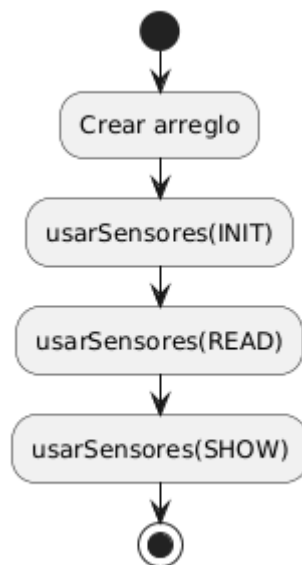


En la nueva implementación, estas operaciones fueron delegadas a la función `usarSensores()`.

```
usarSensores(sensores, INIT);  
usarSensores(sensores, READ);  
usarSensores(sensores, SHOW);
```

Esto permite centralizar la lógica del sistema y mejorar la organización del código.

Implementación modificada:



La principal diferencia entre ambas versiones es que la nueva estructura presenta una mayor modularidad.

La función `main()` pasa a encargarse únicamente de coordinar la secuencia de ejecución, mientras que las operaciones específicas son ejecutadas por la función `usarSensores()`.

2. Parte 2

2.1. Hito 1

P1: Retorna la cantidad de tiempo de CPU que ha utilizado el programa desde que comenzó a ejecutarse.

P2: Almacenan una cantidad de ticks del reloj del procesador, por lo que no tienen ni medida ni signo.

P3: Porque clock() devuelve ticks y no segundos, y CLOCKS_PER_SEC dice cuantos ticks hay en un segundo, La división convierte los ticks a segundos (unidad de tiempo real).

P4: Realiza una conversión de tipo explícita (entero a decimal).

P5: Se utiliza para que la división se realice usando números decimales, ya que de lo contrario, el resultado de la división será un número entero, lo que haría que se perdiera la precisión del resultado obtenido con los decimales.

2.2. Hito 2

La función cicloPrueba() fue diseñada con el objetivo de medir el tiempo promedio de ejecución asociado a distintas operaciones realizadas dentro de un ciclo iterativo.

Su implementación permite comparar el costo temporal de operaciones simples realizadas en memoria con operaciones que requieren utilizar el sistema de salida estándar mediante printf().

La función recibe como parámetro una variable del tipo caso_t, la cual define el caso que será evaluado.

Los casos implementados son:

- VACIO: ejecuta un ciclo sin realizar ninguna instrucción
- ESPACIO: imprime un espacio en cada iteración
- ASTERISCO: imprime un asterisco en cada iteración
- NEWLINE: imprime un salto de línea en cada iteración

Para seleccionar la operación correspondiente se utilizó una estructura switch, permitiendo organizar el flujo de ejecución de manera clara.

El proceso comienza registrando el tiempo inicial mediante la función clock().

Posteriormente se ejecuta el ciclo correspondiente al caso recibido.

Una vez finalizada la ejecución, se registra el tiempo final.

La diferencia entre ambos valores representa la cantidad de ciclos de reloj consumidos durante la prueba.

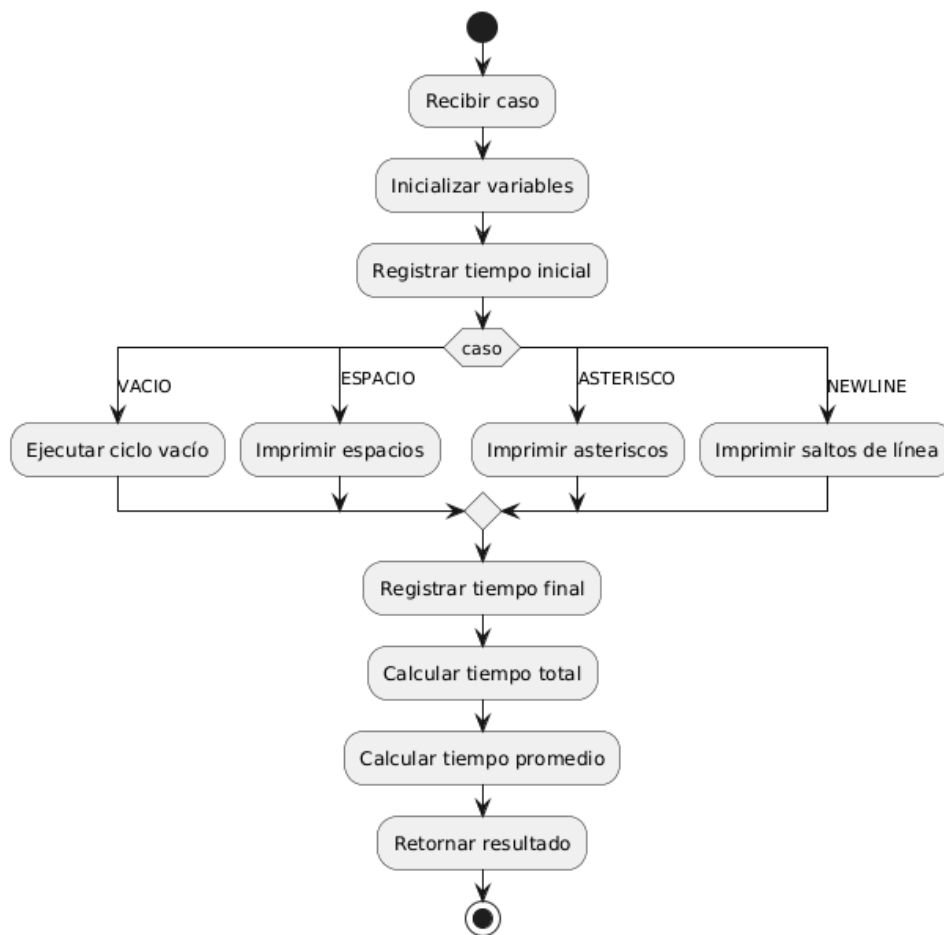
$$(((double)(fin - inicio)) / CLOCKS_PER_SEC) * 1e9$$

En esta expresión, el cast (double) convierte explícitamente el resultado de (fin - inicio) a un valor decimal.

Esto evita la pérdida de precisión durante la división, permitiendo conservar los decimales necesarios para obtener mediciones más exactas.

Finalmente, el tiempo total es dividido por la cantidad de iteraciones ejecutadas, obteniendo así el tiempo promedio por operación.

Diagrama de flujo



El diagrama representa la secuencia completa de ejecución de la función.

Se observa la recepción del caso, la medición inicial, la ejecución condicional mediante switch, el cálculo del tiempo transcurrido y el retorno del tiempo promedio.

2.3. Hito 3

Para realizar las mediciones de tiempo de ejecución, el programa fue ejecutado diez veces de manera independiente.

En cada ejecución se registraron los tiempos promedio obtenidos para los cuatro casos evaluados.

Cada prueba fue realizada utilizando una cantidad de $1e8$ iteraciones.

Este valor fue seleccionado para asegurar que las diferencias temporales entre los distintos casos fueran suficientemente apreciables.

La medición fue realizada utilizando la función `clock()`, la cual permite obtener el tiempo consumido por el procesador durante la ejecución del programa.

Los tiempos obtenidos fueron convertidos a nanosegundos y posteriormente organizados en una tabla para calcular:

- Promedio
- Desviación estándar

Estos indicadores permiten analizar tanto el comportamiento promedio como la variabilidad de los resultados.

Características del entorno de prueba

Computador utilizado: PC de escritorio ensamblado (placa PRIME B460M-A)

Procesador: I5-10400f 2.9 [GHz]

Memoria RAM: 16 [gB] 2600 [MHz]

Sistema operativo: Windows 11

Compilador utilizado: g++

Versión del compilador: MinGW - w64 g++ 13.2.0

Caso	tprom1	tprom2	tprom3	tprom4	tprom5	tprom6
VACIO	0.62	0.58	0.6	0.59	0.61	0.6
ESPACIO	72.4	85.1	68.9	74.2	92.5	70.1
ASTERISCO	75.1	88.3	71.4	76.8	95.2	72.9
NEWLINE	210.5	245.2	198.7	215.3	280.1	205.4

Caso	tprom7	tprom8	tprom9	tprom10	Promedio	Desv. estándar
VACIO	0.57	0.63	0.59	0.6	0.59	0.018
ESPACIO	71.8	88.4	73.5	69.4	76.6	8.65
ASTERISCO	74.3	91.2	76.1	72	79.3	8.76
NEWLINE	208.9	260.6	212.8	201.2	223.8	27.99

A partir de los resultados obtenidos, se espera observar diferencias claras entre los distintos casos evaluados.

El caso VACIO presenta los menores tiempos de ejecución debido a que no involucra operaciones de salida.

Los casos ESPACIO y ASTERISCO presentan tiempos mayores, ya que requieren escritura sobre la salida estándar.

El caso NEWLINE presenta el mayor tiempo promedio, debido a que cada salto de línea puede generar operaciones adicionales relacionadas con la actualización del buffer de salida.

Estas diferencias permiten evidenciar el impacto temporal que tienen las operaciones de entrada/salida en comparación con operaciones ejecutadas exclusivamente en memoria.

La desviación estándar permite además evaluar la estabilidad de las mediciones.

Valores bajos indican consistencia experimental, mientras que valores elevados reflejan mayor variabilidad entre ejecuciones.